



INHA UNIVERSITY TASHKENT

SPRING SEMESTER 2024

SOC 2040

SYSTEM PROGRAMMING

CREDITS/HOURS PER WEEK : 3/3

COURSE TYPE : TECHNICAL CORE SEQUENCE

INSTRUCTOR

DR. A. R. NASEER

2 May 2024

HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG

SOC 2040

SYSTEM PROGRAMMING

WEEK 11 LECTURE 2

PROCESSES

EXCEPTIONAL CONTROL FLOW

CREATING & TERMINATING PROCESSES

Creating and Terminating Processes

From a programmer's perspective, we can think of a process as being in one of three states

Running

- Process is either executing, or waiting to be executed and will eventually be *scheduled* (i.e., chosen to execute) by the kernel

Stopped

- Process execution is *suspended* and will not be scheduled until further notice.
- A process stops as a result of receiving a SIGSTOP, SIGTSTP, SIGTTIN, or SIGTTOU signal, and it remains stopped until it receives a SIGCONT signal, at which point it can begin running again.

Terminated

- **Process is stopped permanently**

A process becomes terminated for one of three reasons:

- (1) receiving a signal whose default action is to terminate the process,
- (2) returning from the main routine, or
- (3) calling the exit function

Terminating Processes

- ★ Process becomes terminated for one of three reasons:
 - Receiving a signal whose default action is to terminate
 - Returning from the `main` routine
 - Calling the `exit` function
- ```
#include <stdlib.h>

void exit(int status)
```
- Terminates with an *exit status* of `status`
  - Convention: normal return status is 0, nonzero on error
  - Another way to explicitly set the exit status is to return an integer value from the main routine
- ★ **`exit` is called once but never returns.**

# Creating Processes

- ❑ **Parent process** creates a new running **child process** by calling **fork**

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
int fork(void)
```

- Returns 0 to the child process, child's PID to parent process

- ❑ **Child is *almost* identical to parent:**

- Child get an identical (but separate) copy of the parent's virtual address space.
- Child gets identical copies of the parent's open file descriptors
- Child has a different PID than the parent

- **fork** is interesting (and often confusing) because it is called **once** but returns **twice**: once in the calling process (the parent), and once in the newly created child process.

- In the parent, fork returns the PID of the child.
- In the child, fork returns a value of 0.
- Since the PID of the child is always non zero, the return value provides an unambiguous way to tell whether the program is executing in the parent or the child.

# System Call Error Handling

- ★ On error, Linux system-level functions typically return **-1** and set global variable **errno** to indicate cause.
- ★ Hard and fast rule:
  - You must check the return status of every system-level function
  - Only exception is the handful of functions that return `void`
- ★ Example:

```
if ((pid = fork()) < 0) {
 fprintf(stderr, "fork error: %s\n", strerror(errno));
 exit(0);
}
```

# Error-reporting functions

- ★ Can simplify somewhat using an *error-reporting function*:

```
void unix_error(char *msg) /* Unix-style error */
{
 fprintf(stderr, "%s: %s\n", msg, strerror(errno));
 exit(0);
}
```

```
if ((pid = fork()) < 0)
 unix_error("fork error");
```



# Error-handling Wrappers

- ★ We simplify the code we present to you even further by using Stevens-style error-handling wrappers:

```
pid_t Fork(void)
{
 pid_t pid;

 if ((pid = fork()) < 0)
 unix_error("Fork error");
 return pid;
}
```

```
pid = Fork();
```

# Obtaining Process IDs

Unix provides a number of system calls for manipulating processes from C programs.

- Each process has a unique positive (nonzero) process ID (PID).
- The `getpid` function returns the PID of the calling process.
- The `getppid` function returns the PID of its parent (i.e., the process that created the calling process).

```
#include <sys/types.h>
```

```
#include <unistd.h>
```

```
pid_t getpid(void)
```

- Returns PID of current process

```
pid_t getppid(void)
```

- Returns PID of parent process

The `getpid` and `getppid` routines return an integer value of type `pid_t`, which on Linux systems is defined in `types.h` as an `int`.

# fork Example

```
int main()
{
 pid_t pid;
 int x = 1;

 pid = Fork();
 if (pid == 0) { /* Child */
 printf("child : x=%d\n", ++x);
 exit(0);
 }
 /* Parent */
 printf("parent: x=%d\n", --x);
 exit(0);
}
```

*fork.c*

```
$. /fork
parent: x=0
child : x=2
```

## ■ Shared open files

- `stdout` is the same in both parent and child

## ■ Call once, return twice

- once to the parent and once to the newly created child

## ■ Concurrent execution

- The parent and the child are separate processes that run concurrently.
- The instructions in their logical control flows can be interleaved by the kernel in an arbitrary way
- Can't predict execution order of parent and child

## ■ Duplicate but separate address space

- Each process has the same user stack, the same local variable values, the same heap, the same global variable values, and the same code.
- However, since the parent and the child are separate processes, they each have their own private address spaces.
- Thus, in our example program, local variable `x` has a value of 1 in both the parent and the child when the `fork` function returns INITIALLY.
- Any subsequent changes that a parent or child makes to `x` are private and are not reflected in the memory of the other process.
- This is why the variable `x` has different values in the parent and child when they call their respective `printf` statements.
- `x` has a value of 1 when `fork` returns in parent and child
- Subsequent changes to `x` are independent

# Modeling fork with Process Graphs

- ★ *A process graph* is a useful tool for capturing the partial ordering of statements in a concurrent program:
  - Each vertex is the execution of a statement
  - $a \rightarrow b$  means  $a$  happens before  $b$
  - Edges can be labeled with current value of variables
  - `printf` vertices can be labeled with output
  - Each graph begins with a vertex with no inedges
- ★ Any *topological sort* of the graph corresponds to a feasible total ordering.
  - Total ordering of vertices where all edges point from left to right

# Process Graph Example

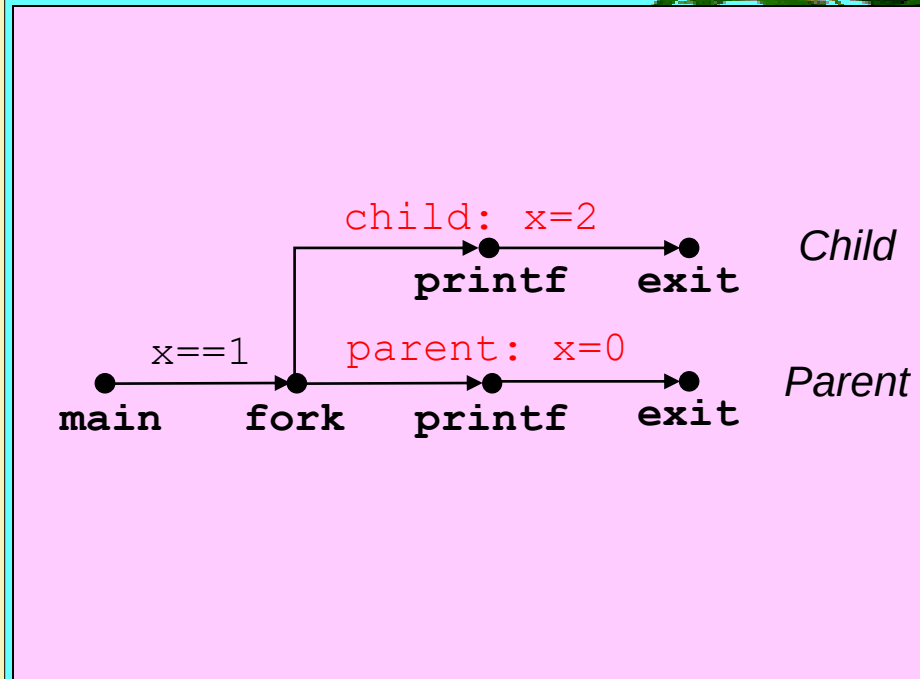
Sketch the process graph, where each horizontal arrow corresponds to a process that executes instructions from left to right, and each vertical arrow corresponds to the execution of a fork function.

```
int main()
{
 pid_t pid;
 int x = 1;

 pid = Fork();
 if (pid == 0) { /* Child */
 printf("child : x=%d\n", ++x);
 exit(0);
 }

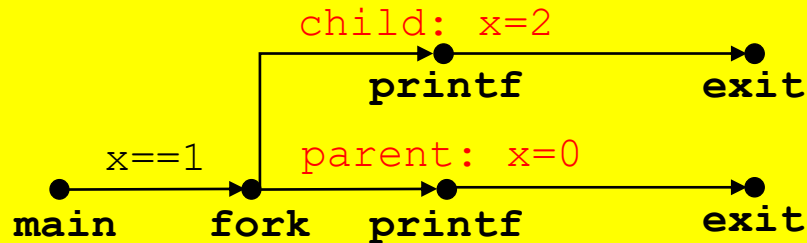
 /* Parent */
 printf("parent: x=%d\n", --x);
 exit(0);
}
```

*fork.c*

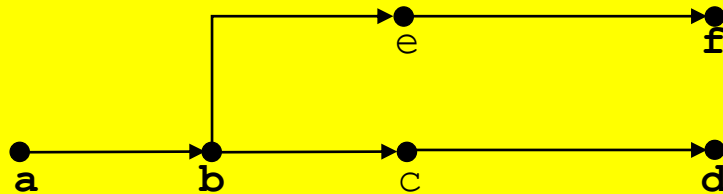


# Interpreting Process Graphs

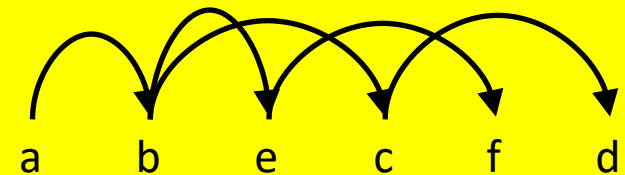
★ Original graph:



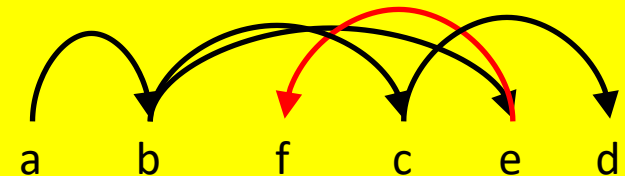
★ Relabelled graph:



Feasible total ordering:



Infeasible total ordering:



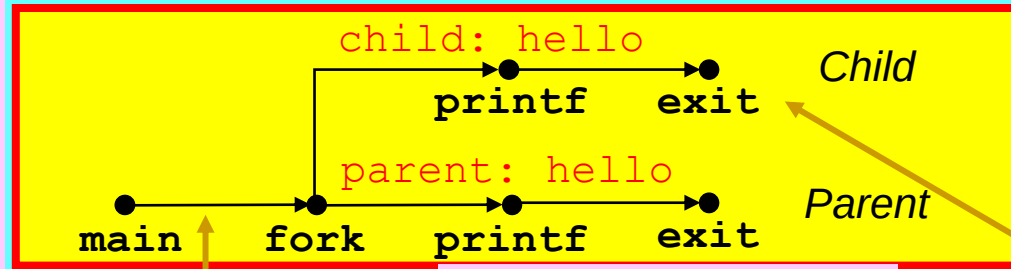
# Modeling fork with Process Graphs

Examples of fork programs and their process graphs

(a) Calls fork once

```
int main()
{
 fork();
 printf("hello\n");
 exit(0);
}
```

fork1.c



PARENT  
fork();  
printf("hello\n");  
exit(0);

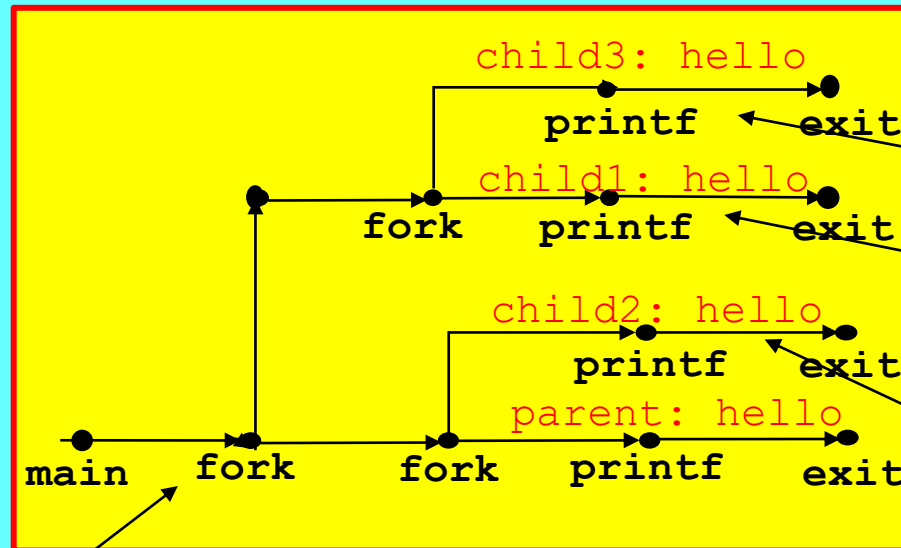
Prints two output lines  
hello ← Parent  
hello ← Child

CHILD  
printf("hello\n");  
exit(0);

(b) Calls fork twice

```
int main()
{
 fork();
 fork();
 printf("hello\n");
 exit(0);
}
```

fork2.c



fork(); PARENT  
fork();  
printf("hello\n");  
exit(0);

Prints four output lines

CHILD3  
printf("hello\n");  
exit(0);

CHILD1  
fork();  
printf("hello\n");  
exit(0);

CHILD2  
printf("hello\n");  
exit(0);

# Modeling fork with Process Graphs

Examples of fork programs and their process graphs

(c) Calls fork three times

```
int main()
{
 fork();
 fork();
 fork();
 printf("hello\n");
 exit(0);
}
```

fork3.c

```
fork(); CHILD1
fork();
printf("hello\n");
exit(0);
```

```
fork(); CHILD2
printf("hello\n");
exit(0);
```

```
fork(); PARENT
fork();
fork();
printf("hello\n");
exit(0);
```

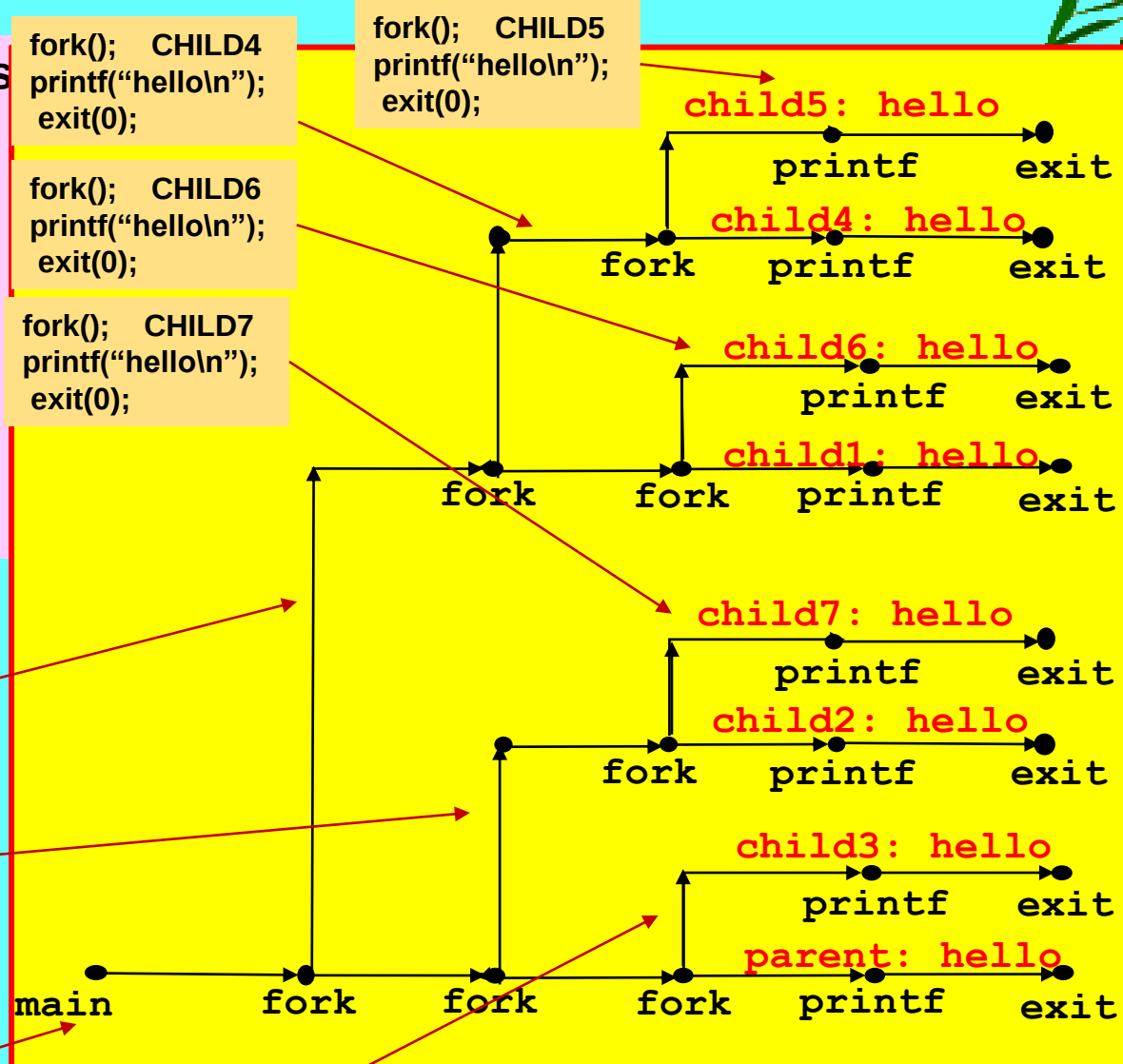
```
CHILD3
printf("hello\n");
exit(0);
```

```
fork(); CHILD4
printf("hello\n");
exit(0);
```

```
fork(); CHILD6
printf("hello\n");
exit(0);
```

```
fork(); CHILD7
printf("hello\n");
exit(0);
```

```
fork(); CHILD5
printf("hello\n");
exit(0);
```



Prints eight output lines



# fork Example: Two consecutive forks

```
int main()
{
 printf("L0\n");
 fork();
 printf("L1\n");
 fork();
 printf("Bye\n");
}
```

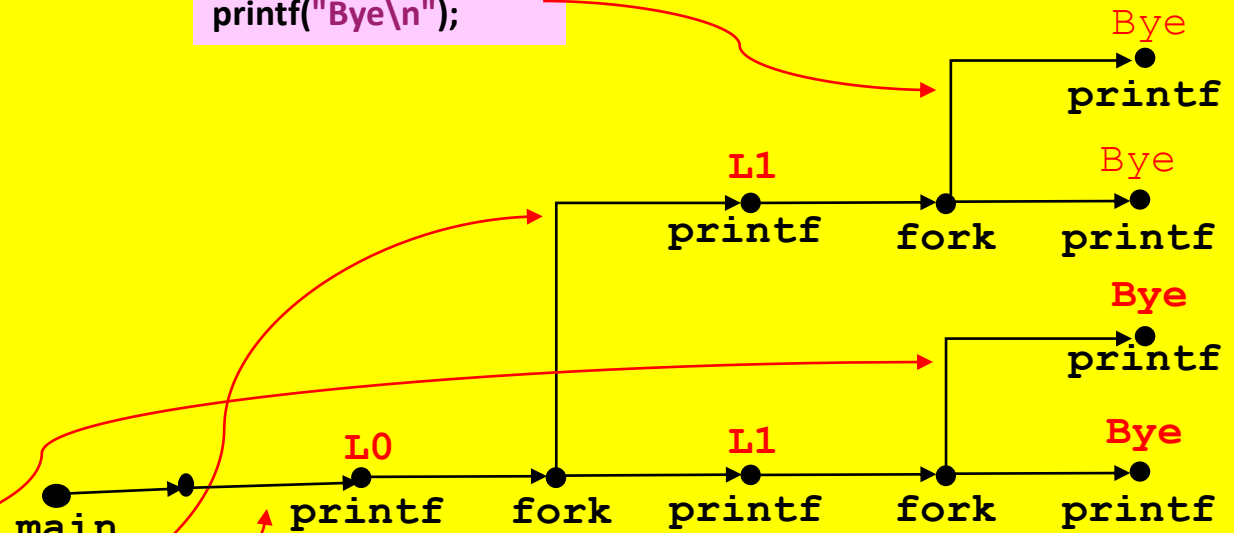
*fork4.c*

CHILD2  
printf("Bye\n");

printf("L1\n"); CHILD1  
fork();  
printf("Bye\n");

printf("L0\n"); PARENT  
fork();  
printf("L1\n");  
fork();  
printf("Bye\n");

fork(); CHILD3  
printf("Bye\n");



Feasible output:

L0  
L1  
L1  
Bye  
Bye  
Bye  
Bye

Infeasible output:

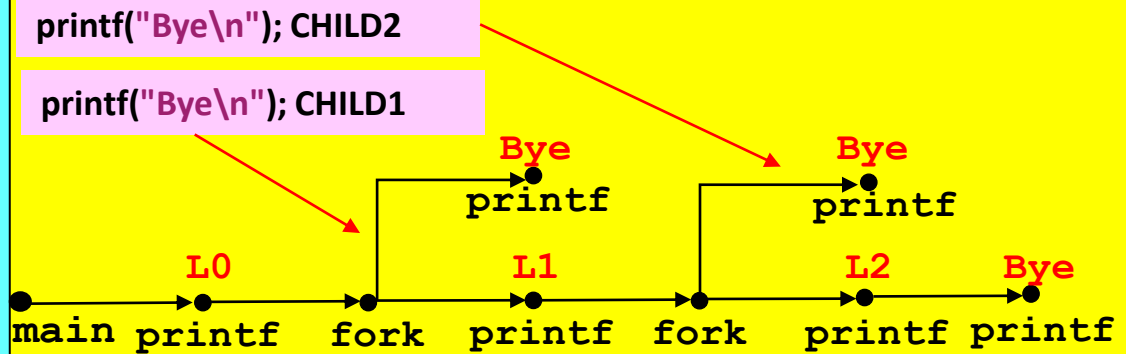
L0  
Bye  
L1  
Bye  
L1  
Bye  
Bye

# fork Example: Nested forks in parent

```
int main()
{
 printf("L0\n");
 if (fork() != 0) {
 printf("L1\n");
 if (fork() != 0) {
 printf("L2\n");
 }
 }
 printf("Bye\n");
}
```

*fork5.c*

```
printf("L0\n"); PARENT
if (fork() != 0) {
 printf("L1\n");
 if (fork() != 0) {
 printf("L2\n");
 }
}
printf("Bye\n");
```



Feasible output:

L0  
L1  
Bye  
Bye  
L2  
Bye

Infeasible output:

L0  
Bye  
L1  
Bye  
Bye  
L2

# fork Example: Nested forks in children

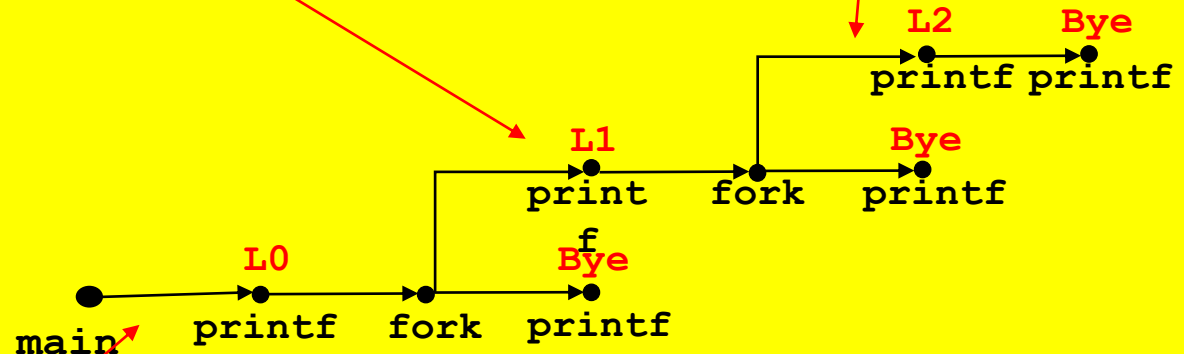
```
int main()
{
 printf("L0\n");
 if (fork() == 0) {
 printf("L1\n");
 if (fork() == 0) {
 printf("L2\n");
 }
 }
 printf("Bye\n");
}
```

*fork6.c*

```
printf("L0\n"); PARENT
fork()
printf("Bye\n");
```

```
printf("L1\n"); CHILD1
fork()
printf("Bye\n");
```

```
printf("L2\n"); CHILD2
printf("Bye\n");
```



Feasible output:

L0  
Bye  
L1  
L2  
Bye  
Bye

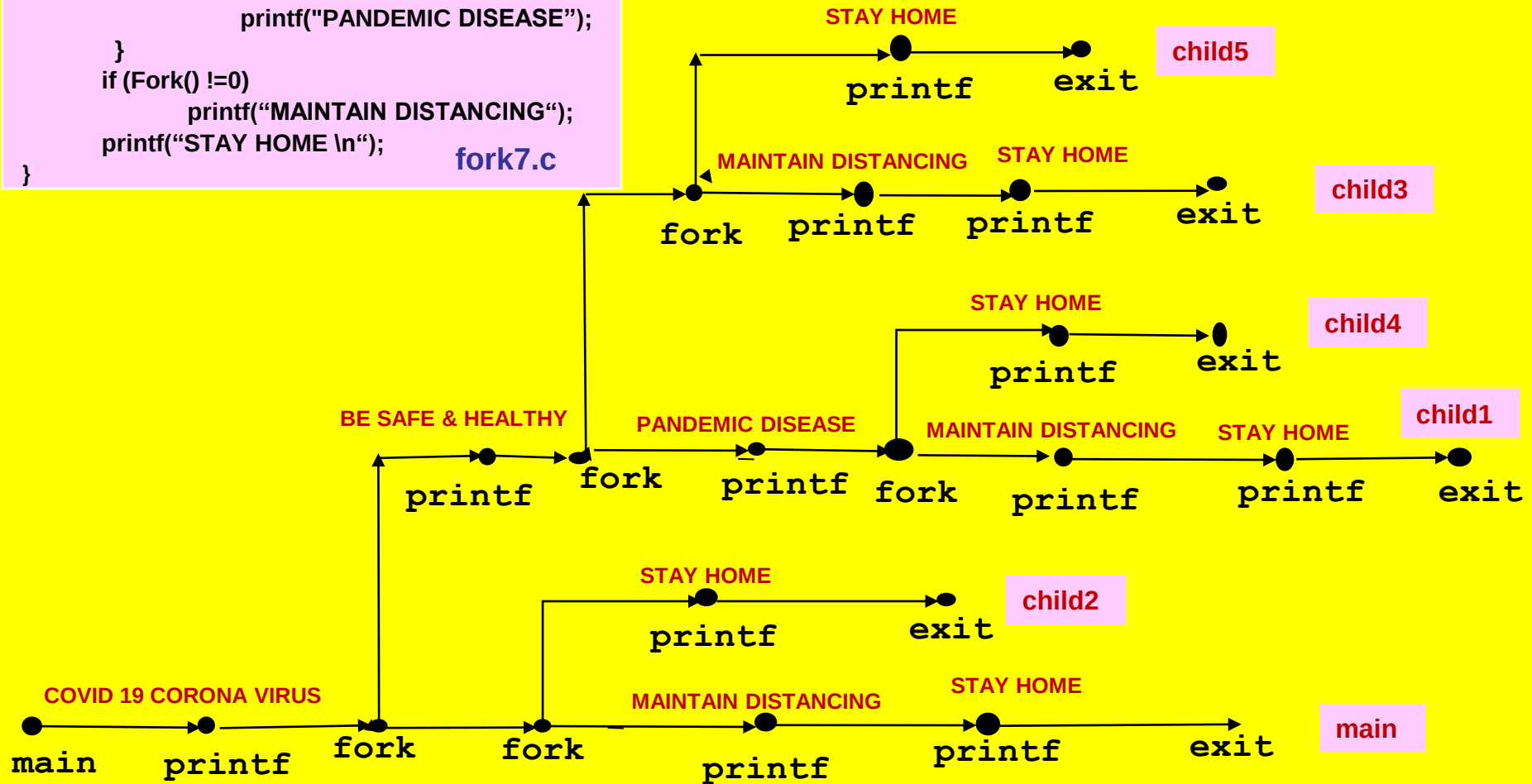
Infeasible output:

L0  
Bye  
L1  
Bye  
Bye  
L2

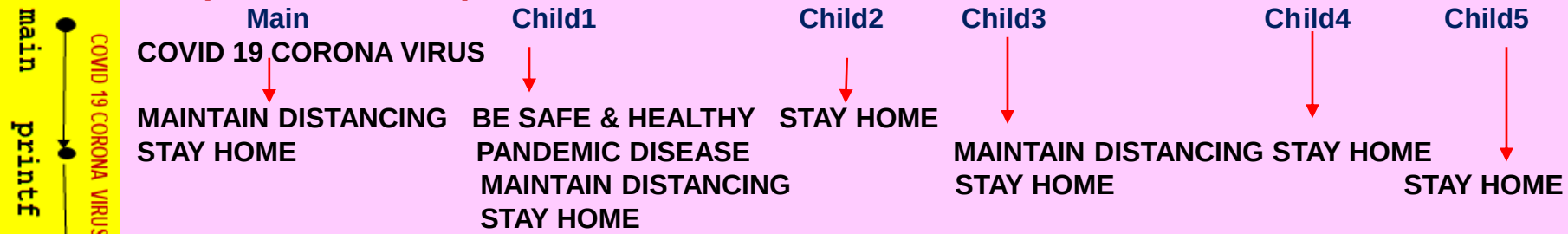
# fork Example: Nested forks in children

```
int main()
{
 pid_t pid;
 printf("COVID 19 CORONA VIRUS ");
 if (fork() == 0)
 { printf("BE SAFE & HEALTHY");
 if (Fork() != 0)
 printf("PANDEMIC DISEASE");
 }
 if (Fork() !=0)
 printf("MAINTAIN DISTANCING");
 printf("STAY HOME \n");
}
```

fork7.c



**Feasible output can be generated by going through the outputs generated by the parent & child processes**

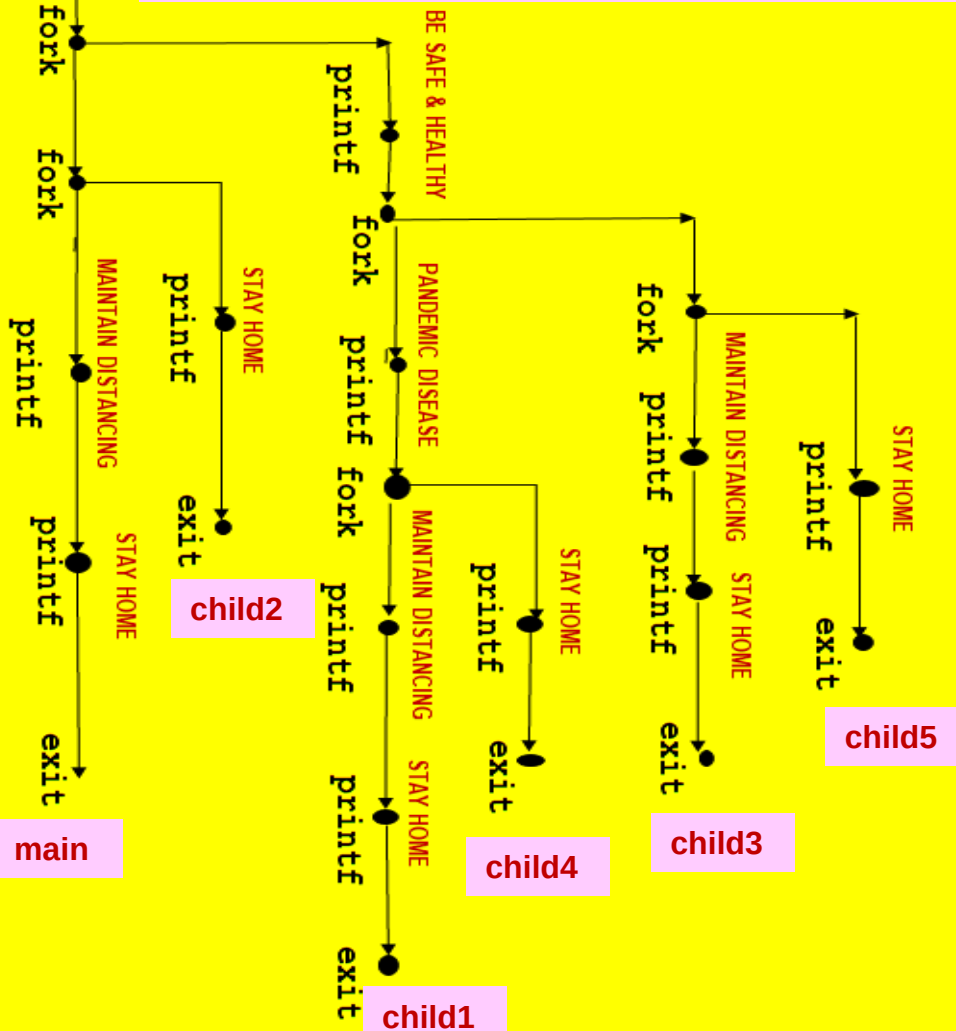


**One possible feasible output:**

COVID 19 CORONA VIRUS  
 BE SAFE & HEALTHY  
 MAINTAIN DISTANCING  
 STAY HOME  
 PANDEMIC DISEASE  
 STAY HOME  
 MAINTAIN DISTANCING  
 STAY HOME  
 MAINTAIN DISTANCING  
 STAY HOME  
 STAY HOME  
 STAY HOME

**One infeasible output:**

COVID 19 CORONA VIRUS  
 BE SAFE & HEALTHY  
 MAINTAIN DISTANCING  
 STAY HOME  
 STAY HOME  
 PANDEMIC DISEASE  
 MAINTAIN DISTANCING  
 STAY HOME  
 STAY HOME  
 STAY HOME  
 STAY HOME  
 MAINTAIN DISTANCING



# Provide all the outputs of the parent and child processes

Feasible output can be generated by going through the outputs generated by the parent & child processes

| Main                             | Child1                                                                    | Child2    | Child3                           | Child4    | Child5    |
|----------------------------------|---------------------------------------------------------------------------|-----------|----------------------------------|-----------|-----------|
| COVID 19 CORONA VIRUS            |                                                                           |           |                                  |           |           |
| ↓                                | ↓                                                                         | ↓         | ↓                                | ↓         | ↓         |
| MAINTAIN DISTANCING<br>STAY HOME | BE SAFE & HEALTHY<br>PANDEMIC DISEASE<br>MAINTAIN DISTANCING<br>STAY HOME | STAY HOME | MAINTAIN DISTANCING<br>STAY HOME | STAY HOME | STAY HOME |

One possible feasible output:

COVID 19 CORONA VIRUS  
BE SAFE & HEALTHY  
MAINTAN DISTANCING  
STAY HOME  
PANDEMIC DISEASE  
STAY HOME  
MAINTAIN DISTANCING  
STAY HOME  
MAINTAIN DISTANCING  
STAY HOME  
STAY HOME  
STAY HOME  
STAY HOME

Another possible feasible output:

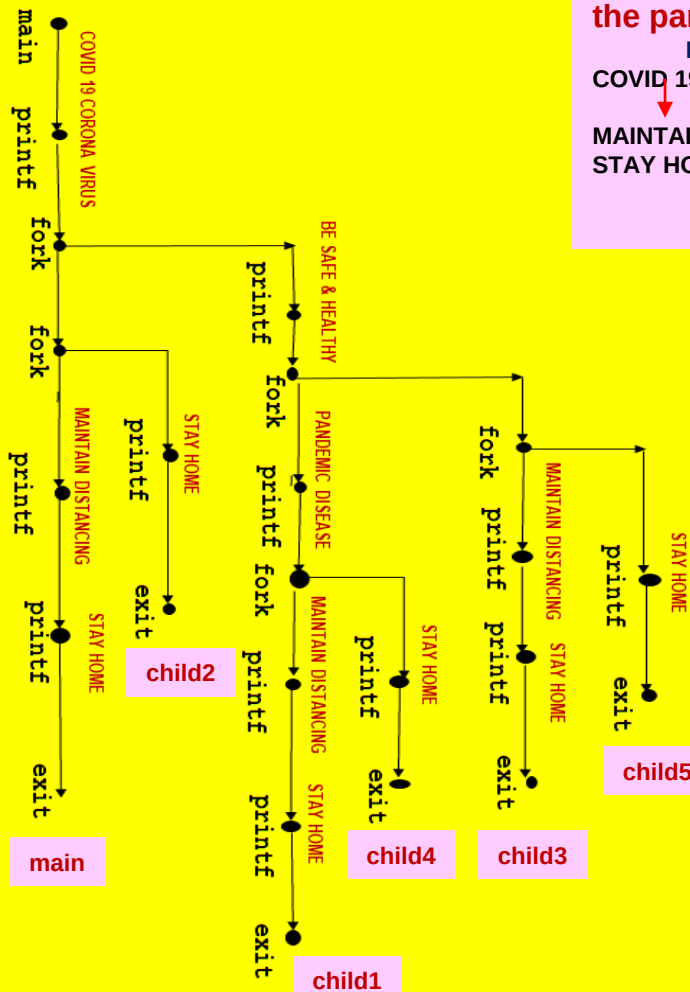
COVID 19 CORONA VIRUS  
MAINTAN DISTANCING  
BE SAFE & HEALTHY  
STAY HOME  
STAY HOME  
PANDEMIC DISEASE  
MAINTAIN DISTANCING  
STAY HOME  
MAINTAIN DISTANCING  
STAY HOME  
STAY HOME  
STAY HOME  
STAY HOME

One infeasible output:

COVID 19 CORONA VIRUS  
MAINTAN DISTANCING  
BE SAFE & HEALTHY  
STAY HOME  
STAY HOME  
PANDEMIC DISEASE  
MAINTAIN DISTANCING  
STAY HOME  
STAY HOME  
STAY HOME  
STAY HOME  
STAY HOME  
MAINTAIN DISTANCING

Another infeasible output:

COVID 19 CORONA VIRUS  
PANDEMIC DISEASE  
MAINTAN DISTANCING  
BE SAFE & HEALTHY  
STAY HOME  
STAY HOME  
MAINTAIN DISTANCING  
STAY HOME  
STAY HOME  
STAY HOME  
MAINTAIN DISTANCING  
STAY HOME

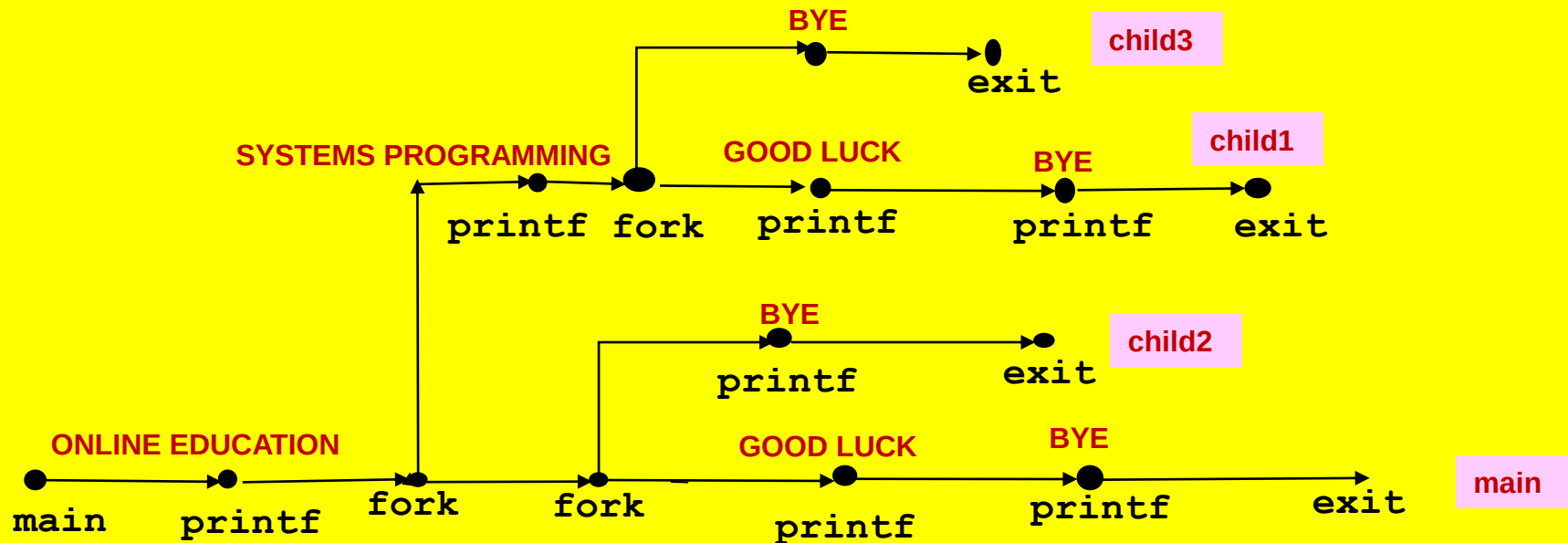


# fork Example: Nested forks in children

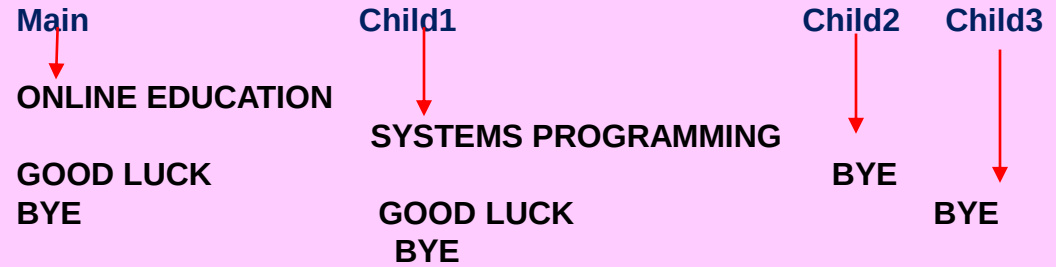
Draw the corresponding Process Graph for the program clearly showing the parent and child processes execution from creation to termination. Provide all the outputs of the parent and child processes

```
int main()
{
 printf("ONLINE EDUCATION");
 if (fork() == 0)
 printf("SYSTEMS PROGRAMMING");
 if (fork() != 0)
 printf("GOOD LUCK");
 printf("BYE \n");
}
```

fork8.c



Feasible output can be generated by going through the outputs generated by the parent & child processes



One possible feasible output:

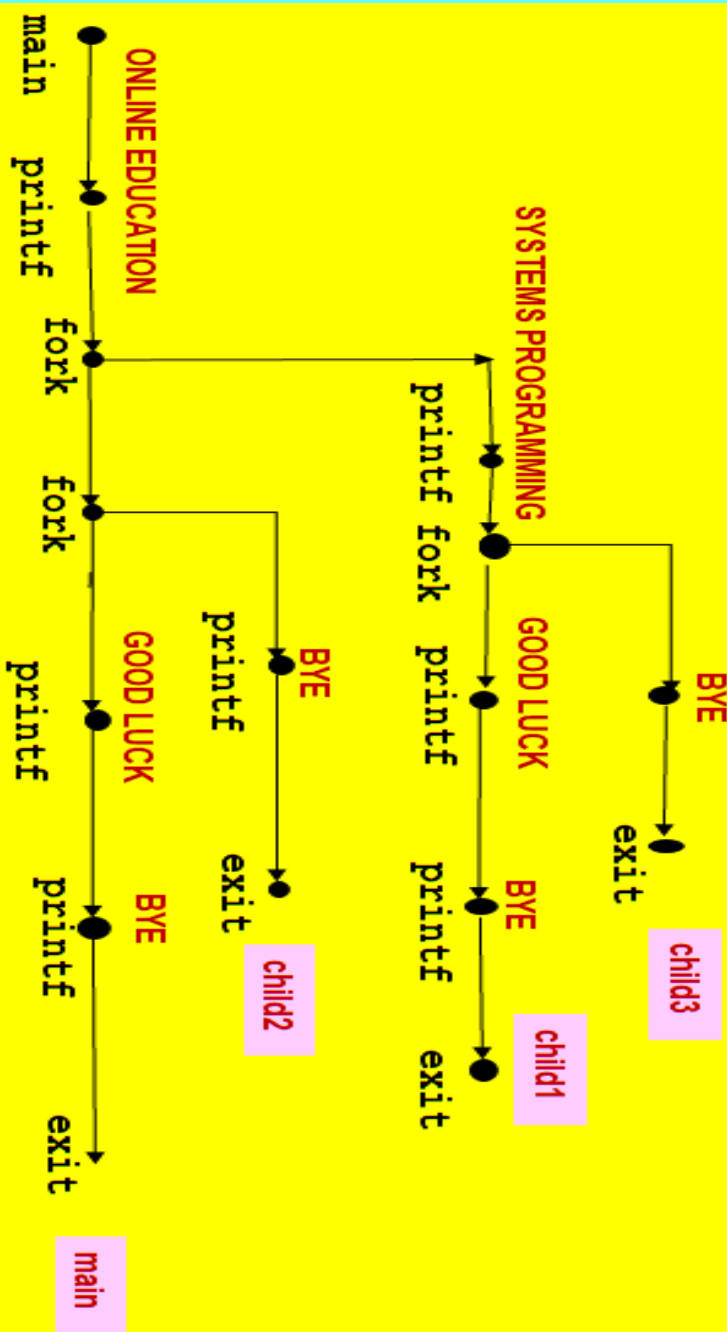
ONLINE EDUCATION  
SYSTEMS PROGRAMMING  
GOOD LUCK  
BYE  
GOOD LUCK  
BYE  
BYE  
BYE

Another possible feasible output:

ONLINE EDUCATION  
SYSTEMS PROGRAMMING  
GOOD LUCK  
BYE  
BYE  
GOOD LUCK  
BYE  
BYE

One infeasible output:

ONLINE EDUCATION  
SYSTEMS PROGRAMMING  
GOOD LUCK  
BYE  
BYE  
BYE  
BYE  
GOOD LUCK





# **EXCEPTIONAL CONTROL FLOW**

**REAPING CHILD  
PROCESSES**

# Reaping Child Processes



## \* Idea

- When process terminates for some reason, it still consumes system resources; kernel does not remove it from the system immediately.

Examples: Exit status, various OS tables

- Instead, the process is kept around in a terminated state until it is reaped by its parent.
- When the parent reaps the terminated child, the kernel passes the child's exit status to the parent, and then discards the terminated process, at which point it ceases to exist.
- A terminated process that has not yet been reaped is called a **“zombie”**
  - \* Living corpse, half alive and half dead

## \* Reaping

- Performed by parent on terminated child (using `wait` or `waitpid`)
- Parent is given exit status information
- Kernel then deletes zombie child process

## \* What if parent doesn't reap?

- If any parent terminates without reaping a child, then the orphaned child will be reaped by `init` process (`pid == 1`)
- So, only need explicit reaping in long-running processes
  - \* e.g., shells and servers



# Zombie Example

```
void main() {
 if (fork() == 0) {
 /* Child */
 printf("Terminating Child, PID = %d\n", getpid());
 exit(0);
 } else {
 printf("Running Parent, PID = %d\n", getpid());
 while (1)
 ; /* Infinite loop */
 }
}
```

*fork8.c*

```
linux> ./fork8&
[1] 6639
Running Parent, PID = 6639
Terminating Child, PID = 6640
linux> ps
 PID TTY TIME CMD
 6585 ttyp9 00:00:00 tcsh
 6639 ttyp9 00:00:03 fork8
 6640 ttyp9 00:00:00 fork8 <defunct>
 6641 ttyp9 00:00:00 ps
linux> kill 6639
[1] Terminated
linux> ps
 PID TTY TIME CMD
 6585 ttyp9 00:00:00 tcsh
 6642 ttyp9 00:00:00 ps
```

- \* `ps` shows child process as “defunct” (i.e., a zombie)
- \* Killing parent allows child to be reaped by `init`

# Non-terminating Child Example

```
linux> ./fork9
Terminating Parent, PID = 6675
Running Child, PID = 6676
linux> ps
 PID TTY TIME CMD
 6585 ttyp9 00:00:00 tcsh
 6676 ttyp9 00:00:06 fork9
 6677 ttyp9 00:00:00 ps
linux> kill 6676
linux> ps
 PID TTY TIME CMD
 6585 ttyp9 00:00:00 tcsh
 6678 ttyp9 00:00:00 ps
```

```
void main()
{
 if (fork() == 0) {
 /* Child */
 printf("Running Child, PID = %d\n",
 getpid());
 while (1)
 ; /* Infinite loop */
 } else {
 printf("Terminating Parent, PID = %d\n",getpid());
 exit(0);
 }
}
```

*fork9.c*

★ Child process still active even though parent has terminated

★ Must kill child explicitly, or else will keep running indefinitely

# **waitpid: Waiting for a Specific Process**

A process waits for its children to terminate or stop by calling the `waitpid` function

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
pid_t waitpid(pid_t pid, int *statusp, int options)
```

Returns PID of child if OK, 0 (if WNOHANG) or -1 on error

- **Suspends current process until specific process terminates**
- **The waitpid function is complicated.**
- **By default (when options = 0), waitpid suspends execution of the calling process until a child process in its wait set terminates.**
- **If a process in the wait set has already terminated at the time of the call, then waitpid returns immediately.**
- **In either case, waitpid returns the PID of the terminated child that caused waitpid to return, and the terminated child is removed from the system.**

# **waitpid: Waiting for a Specific Process**

## **❑ Determining the Members of the Wait Set**

**The members of the wait set are determined by the pid argument:**

- If  $\text{pid} > 0$ , then the wait set is the singleton child process whose process ID is equal to pid.**
- If  $\text{pid} = -1$ , then the wait set consists of all of the parent's child processes.**
- The waitpid function also supports other kinds of wait sets, involving Unix process groups**

# waitpid: Waiting for a Specific Process

## ❑ Modifying the Default Behavior

The default behaviour can be modified by setting options to various combinations of the WNOHANG and WUNTRACED constants:

### WNOHANG

- ★ Return immediately (with a return value of 0) if none of the child processes in the wait set has terminated yet.
- ★ The default behavior suspends the calling process until a child terminates.
- ★ This option is useful in those cases where you want to do useful work while waiting for a child to terminate.

# **waitpid: Waiting for a Specific Process**

## **❑ Modifying the Default Behavior**

### **WUNTRACED**

- ★ Suspend execution of the calling process until a process in the wait set becomes either terminated or stopped.
- ★ Return the PID of the terminated or stopped child that caused the return.
- ★ The default behavior returns only for terminated children.
- ★ This option is useful when you want to check for both terminated and stopped children.

### **WNOHANG | WUNTRACED :**

- ★ Return immediately, with a return value of 0, if none of the children in the wait set has stopped or terminated, or with a return value equal to the PID of one of the stopped or terminated children.



# **waitpid: Waiting for a Specific Process**

## ❑ **Checking the Exit Status of a Reaped Child**

- ★ If the status argument is non-NULL, then **waitpid** encodes status information about the child that caused the return in the status argument.
- ★ The **wait.h** header file defines several macros for interpreting the status argument:

### **WIFEXITED(status):**

- ★ Returns true if the child terminated normally, via a call to exit or a return.

### **WEXITSTATUS(status):**

- ★ Returns the exit status of a normally terminated child. This status is only defined if WIFEXITED returned true.

### **WIFSIGNALED(status):**

Returns true if the child process terminated because of a signal that was not caught.

# **waitpid: Waiting for a Specific Process**

- ❑ **Checking the Exit Status of a Reaped Child**

## **WTERMSIG(status):**

Returns the number of the signal that caused the child process to terminate.

This status is only defined if WIFSIGNALED(status) returned true.

## **WIFSTOPPED(status):**

Returns true if the child that caused the return is currently stopped.

## **WSTOPSIG(status):**

Returns the number of the signal that caused the child to stop.

This status is only defined if WIFSTOPPED(status) returned true.

# **waitpid: Waiting for a Specific Process**

## ❑ **Error Conditions**

- ★ If the calling process has no children, then **waitpid** returns **-1** and sets **errno** to **ECHILD**.
- ★ If the **waitpid** function was interrupted by a signal, then it returns **-1** and sets **errno** to **EINTR**.
- ★ Constants such as **WNOHANG** and **WUNTRACED** are defined by system header files.
- ★ For example, **WNOHANG** and **WUNTRACED** are defined (indirectly) by the **wait.h** header file:

```
/* Bits in the third argument to 'waitpid'. */
#define WNOHANG 1 /* Don't block waiting. */
#define WUNTRACED 2 /* Report status of stopped children. */
```
- ★ In order to use these constants, you must include the **wait.h** header file in your code:

```
#include <sys/wait.h>
```
- ★ In order to check return codes such as **ECHILD** and **EINTR**, you must include **errno.h**.

```
#include <errno.h>
```

# waitpid: Waiting for Processes in no order

```
int main()
{
```

*fork10.c*

```
 int status, i;
 pid_t pid;
```

Using the waitpid function to reap zombie children in no particular order

```
 /* Parent creates N children */
 for (i = 0; i < N; i++)
 if ((pid = Fork()) == 0) /* Child */
 exit(100+i);
```

```
 /* Parent reaps N children in no particular order */
 while ((pid = waitpid(-1, &status, 0)) > 0)
 {
 if (WIFEXITED(status))
 printf("child %d terminated normally with exit status=%d\n", pid,
 WEXITSTATUS(status));

 else
 printf("child %d terminated abnormally\n", pid);
 }
 /* The only normal termination is if there are no more children */
 if (errno != ECHILD)
 unix_error("waitpid error");
 exit(0);
}
```

# waitpid: Waiting for Specific Processes in order

```
int main()
{
```

*fork11.c*

```
 int status, i;
 pid_t pid[N], retpid;
```

```
 /* Parent creates N children */
 for (i = 0; i < N; i++)
 if ((pid[i] = Fork()) == 0) /* Child */
 exit(100+i);
```

Using the waitpid function to reap zombie children in particular order i.e., from Child 0 to Child N-1

```
 /* Parent reaps N children in particular order */
 i = 0;
 while ((retpid = waitpid(pid[i++], &status, 0)) > 0) {
 {
 if (WIFEXITED(status))
 printf("child %d terminated normally with exit status=%d\n", retpid,
 WEXITSTATUS(status));

 else
 printf("child %d terminated abnormally\n", retpid);
 }
 /* The only normal termination is if there are no more children */
 if (errno != ECHILD)
 unix_error("waitpid error");
 exit(0);
}
```

# waitpid: Waiting for a Specific Process

```
void main() {
 pid_t pid[N];
 int i;
 int child_status;
```

*fork12.c*

```
 for (i = 0; i < N; i++)
 if ((pid[i] = fork()) == 0)
 exit(100+i); /* Child */
```

*/\* Using waitpid to reap zombie children in the reverse order of their creation i.e., from child N-1 to child 0\*/*

```
 for (i = N-1; i >= 0; i--)
 {
 pid_t wpid = waitpid(pid[i], &child_status, 0);
 if (WIFEXITED(child_status))
 printf("Child %d terminated with exit status %d\n", wpid,
 WEXITSTATUS(child_status));
 else
 printf("Child %d terminate abnormally\n", wpid);
 }
 /* The only normal termination is if there are no more children */
 if (errno != ECHILD)
 unix_error("waitpid error");
 exit(0);
}
```

# **wait: Synchronizing with Children**

- ★ Parent reaps a child by calling the `wait` function, a simpler version of `waitpid`

```
#include <sys/types.h>
```

```
#include <sys/wait.h>
```

```
pid_t wait(int *statusp);
```

Returns PID of child if OK or -1 on error

- ★ Calling `wait(&status)` is equivalent to calling `waitpid(-1, &status, 0)`.

Suspends current process until one of its children terminates

- Return value is the `pid` of the child process that terminated
- If `statusp != NULL`, then the integer it points to will be set to a value that indicates reason the child terminated and the exit status:
  - ★ Checked using macros defined in `wait.h`
    - `WIFEXITED`, `WEXITSTATUS`, `WIFSIGNALED`, `WTERMSIG`, `WIFSTOPPED`, `WSTOPSIG`, `WIFCONTINUED`

# Another wait Example

- ✱ If multiple children completed, will take in arbitrary order
- ✱ Can use macros WIFEXITED and WEXITSTATUS to get information about exit status

```
void main() {
 pid_t pid[N];
 int i, child_status;

 for (i = 0; i < N; i++)
 if ((pid[i] = fork()) == 0) {
 exit(100+i); /* Child */
 }
 for (i = 0; i < N; i++)
 { /* Parent */
 pid_t wpid = wait(&child_status);
 if (WIFEXITED(child_status))printf("Child %d terminated with exit status %d\n", wpid,
 WEXITSTATUS(child_status));

 else
 printf("Child %d terminate abnormally\n", wpid);
 }
 /* The only normal termination is if there are no more children */
 if (errno != ECHILD)
 unix_error("waitpid error");
 exit(0);
}
```

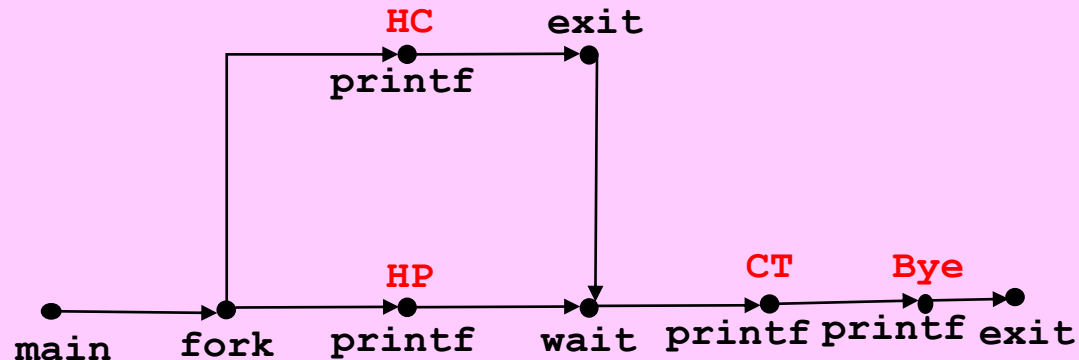
*fork13.c*



# wait: Synchronizing with Children

```
void main() {
 int child_status; fork14.c

 if (fork() == 0) {
 printf("HC: hello from child\n");
 exit(0);
 } else {
 printf("HP: hello from parent\n");
 wait(&child_status);
 printf("CT: child has terminated\n");
 }
 printf("Bye\n");
 exit(0);
}
```



Feasible output:

HC  
HP  
CT  
Bye

Infeasible output:

HP  
CT  
Bye  
HC

# Putting Processes to Sleep

- ★ The sleep function suspends a process for a specified period of time.

```
#include <unistd.h>
```

```
unsigned int sleep(unsigned int secs);
```

**Returns: seconds left to sleep**

- ★ Sleep returns zero if the requested amount of time has elapsed, and the number of seconds still left to sleep otherwise.
- ★ The latter case is possible if the sleep function returns prematurely because it was interrupted by a signal

# Putting Processes to Sleep

- ★ Another function that we will find useful is the **pause function**, which puts the calling function to sleep until a signal is received by the process.

```
#include <unistd.h>
```

```
int pause(void);
```

Always returns -1

# **SOC 2040    SYSTEM PROGRAMMING**

## **Reading Assignments**

### **Chapter 8 of Text Book**

#### **➤ Computer Systems : A Programmer's Perspective**

- Randal E. Bryant and David R. O'Hallaron  
3<sup>rd</sup> Edition, Global Edition   Prentice Hall   2016  
ISBN 10:1-292-10176-8**



# **INHA UNIVERSITY TASHKENT**

## **SPRING SEMESTER 2024**

### **SOC 2040**

### **SYSTEM PROGRAMMING**

**CREDITS/HOURS PER WEEK : 3/3**

**COURSE TYPE : TECHNICAL CORE SEQUENCE**

**INSTRUCTOR**

**DR. A. R. NASEER**

2 May 2024

**HEAD & PROFESSOR OF COMPUTER SCIENCE & ENGG**